

Tiling game

Barchin y Charos están jugando a un juego en una cuadrícula de $2N \times 2M$ casillas cuadradas. Las filas están numeradas 0 al $2N - 1$ de arriba a abajo, y las columnas están numeradas 0 al $2M - 1$ de izquierda a derecha. Para cada $0 \leq i < 2N$ y $0 \leq j < 2M$, denotamos la casilla en la fila i y la columna j por (i, j) .

Barchin le da a Charos $N \cdot M$ **bloques**, uno por uno. Cada bloque es un cuadrado de 2×2 que consta de cuatro **casillas** 1×1 . Barchin ha coloreado cada casilla de cada bloque de blanco o negro, garantizando que **al menos una casilla sea blanca** en cada bloque.

Charos debe colocar cada bloque en la cuadrícula **inmediatamente después de recibirlo**, sin saber qué bloques recibirá más adelante. Los bloques no se pueden rotar. Cada bloque debe colocarse completamente dentro de la cuadrícula, cubriendo exactamente cuatro casillas. Además, la casilla superior izquierda de cada bloque debe cubrir una casilla cuyas coordenadas de fila y columna sean pares. Cada casilla de la cuadrícula puede estar cubierta por un máximo de un bloque.

Barchin gana el juego si, después de colocar cualquier bloque, existe un cuadrado de casillas de 2×2 cubierto por cuatro casillas negras. Formalmente, si las casillas (a, b) , $(a + 1, b)$, $(a, b + 1)$ y $(a + 1, b + 1)$ están todas cubiertas por casillas negras para algún $0 \leq a < 2N - 1$ y $0 \leq b < 2M - 1$, entonces Barchin gana. Los índices a y b no tienen por qué ser pares necesariamente.

Charos gana si coloca todos los $N \cdot M$ bloques sin que Barchin gane en ningún momento. Ten en cuenta que al colocar los $N \cdot M$ bloques se cubrirá completamente la cuadrícula.

Tu tarea consiste en implementar una estrategia para que Charos gane la partida. Se puede demostrar que, bajo las restricciones dadas, Charos siempre puede colocar los bloques de manera que garantice la victoria, independientemente de las coloraciones de los bloques que reciba posteriormente.

Detalles de implementación

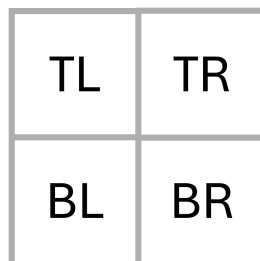
Debes implementar dos funciones:

```
void init(int N, int M)
```

- N : la mitad del número de filas en la cuadrícula.
- M : la mitad del número de columnas en la cuadrícula.
- La función se llama exactamente una vez por cada caso de prueba, al comienzo de la ejecución de tu programa.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : los colores de las casillas superior izquierda, superior derecha, inferior izquierda e inferior derecha del bloque actual, respectivamente, como se muestra en la figura siguiente. Cada valor es 0 (blanco) o 1 (negro).
- La función se llama exactamente $N \cdot M$ veces por caso de prueba, después de la llamada inicial a `init`.



Esta función debe devolver un par de enteros (i, j) , donde i es la coordenada de fila y j es la coordenada de columna de la casilla donde se debe colocar la casilla superior izquierda de este bloque. Tanto i como j deben ser pares, y la región 2×2 cubierta por el bloque no debe superponerse con ningún bloque colocado previamente.

Si `receive_block` devuelve un par que no cumple con estos requisitos, o si después de colocar el bloque un cuadrado de casillas de 2×2 queda completamente cubierto por casillas negras, el evaluador finalizará inmediatamente tu programa y el veredicto para el caso de prueba será `Output isn't correct`.

El comportamiento del evaluador **no es adaptativo**. Esto significa que la secuencia de bloques que Barchin le da a Charos es fija antes de que se llame a `init`.

Restricciones

Para cada bloque, sea S el número de casillas negras entre sus cuatro casillas. Es decir, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ para cada bloque.

Subtareas

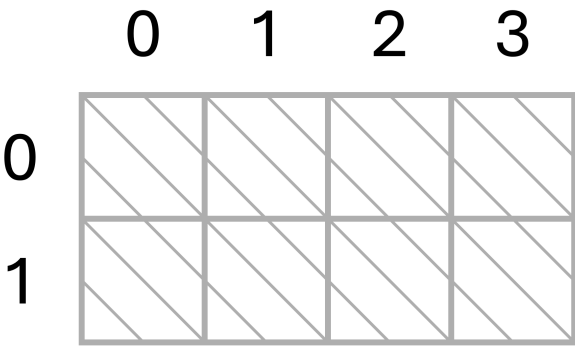
Subtarea	Puntuación	Restricciones adicionales
1	6	$S = 1$ para cada bloque, y $N = 2$.
2	16	$S = 3$ para cada bloque. $N = M$, N es par, y cada una de las cuatro posibles coloraciones de bloques aparece exactamente $\frac{N^2}{4}$ veces.
3	10	$S = 1$ para cada bloque.
4	29	$S \leq 2$ para cada bloque.
5	39	Sin restricciones adicionales.

Ejemplo

Consideremos un juego con $N = 1$ y $M = 2$, de modo que la cuadrícula tiene 2 filas y 4 columnas. El evaluador primero hace la llamada:

```
init(1, 2)
```

Inicialmente, todas las casillas están vacías. La cuadrícula se ve así:



Hay $N \cdot M = 2$ bloques para colocar. Supongamos que Barchin entrega un bloque con tres casillas negras y una casilla blanca en la esquina superior derecha. El evaluador hace la llamada:

```
receive_block(1, 0, 1, 1)
```

Charos decide colocar este bloque en el lado izquierdo de la cuadrícula devolviendo $(0, 0)$.

La cuadrícula ahora se ve así:

	0	1	2	3
0				
1				

A continuación, Barchin entrega otro bloque con tres casillas negras y una casilla blanca en la esquina superior izquierda:

```
receive_block(0, 1, 1, 1)
```

La única casilla restante con fila par y columna par que puede servir como esquina superior izquierda de un bloque de 2×2 es $(0, 2)$, por lo que Charos devuelve $(0, 2)$. La cuadrícula termina así:

	0	1	2	3
0				
1				

Ningún cuadrado 2×2 está completamente cubierto por casillas negras, por lo que Charos ha colocado todos los bloques sin que Barchin haya ganado. Charos gana la partida.

Evaluador de ejemplo

Formato de entrada:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Formato de salida:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Aquí, $R[k]$ y $C[k]$ son los pares de enteros devueltos por la llamada k a `receive_block`.